

Training Bilipschitz Embeddings of Group Quotient Spaces

Nathan Willey

[Project GitHub](#)

The goal of this project is to numerically learn maps from quotient spaces of finite Hilbert spaces to Euclidean space $f : V/G \rightarrow \mathbb{R}^t$ where $G \leq O(V)$. The quotient space V/G is naturally equipped with the metric which for our purposes can be written as

$$d([x], [y]) = \inf_{p \in [x], q \in [y]} \|p - q\| = \min_{g \in G} \|x - gy\|$$

Embeddings will be optimized in order to minimize worst case distortion of the quotient space. This is achieved by training small scale neural networks with various architecture whose loss function is empirical distortion of the training samples. To accomplish this training given only the Hilbert space to embed, an infinite training set is assuming with density according to standard Gaussian entries of the elements of V . Then small batches of training data are sampled, and the models are trained on the empirical distortion of these batches. In this document I outline the specific procedures used to create and train such embeddings. In everything that follows note that $|V|$ denotes the dimension as a real vector space.

1 Architectures

In this section I provide the structure of each of the following embedding maps $f : V/G \rightarrow \mathbb{R}^t$ illustrated in Figure 1. Each map is labeled by an acronym which appears in Table 1. For $x, y \in V$ and $G \leq O(V)$ let $\langle\langle [x], [y] \rangle\rangle = \max_{g \in G} \langle x, gy \rangle$ denote the max filter between x and y with respect to the group G . For $T \in \mathbb{R}^{t \times |V|}$ with rows t_i let

$$(\langle\langle [T], [x] \rangle\rangle) = (\langle\langle [t_1], [x] \rangle\rangle, \dots, \langle\langle [t_k], [x] \rangle\rangle)^T \in \mathbb{R}^k$$

be the max filter of x with the linear map T and group G . Define $\text{sort} : \mathbb{R}^t \rightarrow \mathbb{R}^t$ to be the operator that sorts vectors in a monotonically non-increasing way. Then for $G \leq O(V)$ finite let $\langle\langle [x], [y] \rangle\rangle_{\text{SORT}} = \text{sort}(\langle x, y \rangle, \langle x, g_2 y \rangle \dots, \langle x, g_k y \rangle)$ where $\{e, g_2, \dots, g_k\}$ enumerates the elements of G . Finally for $T \in V^t$ with elements t_i let

$$\langle\langle [T], [x] \rangle\rangle_{\text{SORT}} = (\langle\langle [t_1], [x] \rangle\rangle_{\text{SORT}} \dots \langle\langle [t_n], [x] \rangle\rangle_{\text{SORT}})^T \in \mathbb{R}^{k \cdot t}$$

Embedding Architectures:

RMF: Random Max-Filter Generate a random max-filter bank $T \in V^t$ (standard Gaussian entries).

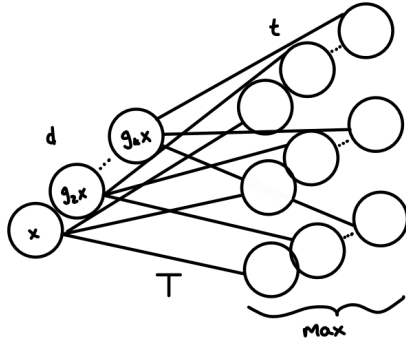
MF: Trained Max Filter Train the entries of a max-filter bank $T \in V^t$. The embedding is given by $\langle\langle [T], [x] \rangle\rangle$.

LRMF: Trained Linear Layer after Random Max-Filter Generate a random max-filter bank $T \in V^t$ (standard Gaussian entries) with rows t_i . Train a linear layer $L \in \mathbb{R}^{t \times t}$. The embedding is given by $L(\langle\langle [T], [x] \rangle\rangle)$.

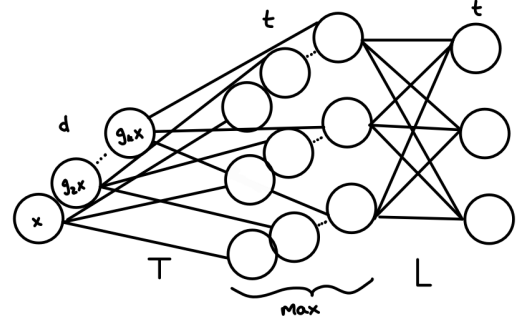
LMF: Trained Linear Layer after Trained Max Filter Train the entries of a max-filter bank $T \in V^t$ as well as a linear layer $L \in \mathbb{R}^{t \times t}$. The embedding is given by $L(\langle\langle [T], [x] \rangle\rangle)$.

SORT: Trained Sort-Based Embedding Train a template matrix $T \in V^{(t/k)}$. The embedding is given by $\langle\langle [T], [x] \rangle\rangle_{\text{SORT}}$.

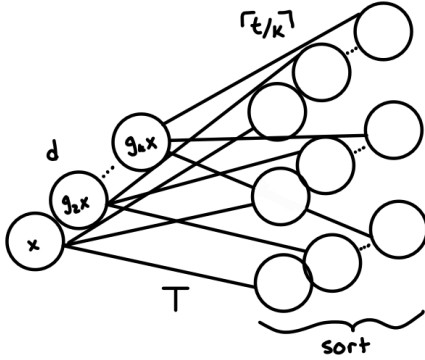
RELU: RELU Network on Orbits Train two linear layers $W \in \mathbb{R}^{kt \times d}$, $L \in \mathbb{R}^{t \times kt}$. The embedding is given by $L(\text{ReLU}(Wx))$. When G is finite we let $k = |G|$ otherwise it is a chosen parameter.



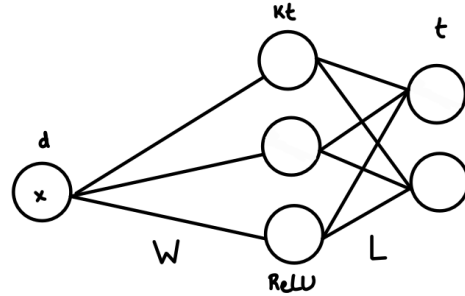
(a) Max-Filtering Embedding



(b) Linear(Max-Filtering) Embedding



(c) SORT-based embedding



(d) Single Layer RELU Network

Figure 1: The various embedding architectures trained. The number above a layer denotes the dimensionality of that layer. A version of a) and b) with T iid Gaussian are also trained.

2 Training

Each map is trained to minimize the distortion of the embedding into its target dimension t . Previously, the models were trained on squared distortion, however this loss function is inherently less stable than distortion since any small values in the denominator of distortion get squared. For this reason, the models are now trained on distortion which provides higher stability. During training, mini-batches of data are randomly sampled according to the standard Gaussian and the training loss is the empirical squared worst-case distortion of the mini-batch given by (β/α) for measured upper Lipschitz constant β and lower Lipschitz constant α . Equivalently we can write the empirical distortion as

$$\text{loss} = \max_{i,j \in 1,\dots,n} \frac{\|f(x_i) - f(x_j)\|}{d([x_i], [x_j])} \cdot \max_{i',j' \in 1,\dots,n} \frac{d([x_{i'}], [x_{j'}])}{\|f(x_{i'}) - f(x_{j'})\|}$$

In order to compute this a quotient distance matrix for each mini-batch is computed and compared to the distance matrix resulting from the network embedding. In practice, we define some small tolerance ϵ and ignore the distortion of any two points whose quotient distance is $< \epsilon$ in order to avoid numerical issues. To compute the max-filter for a **finite** group $G \leq O(V)$ of size k , each data point x is mapped to its orbit $(x, g_2x, \dots, g_kx)$. A shared linear template is then applied across the orbit, and the maximum is taken over the k transformed points. A similar process is used for the SORT embedding, but instead of taking a maximum, the outputs along the group axis are sorted and then concatenated/flattened. Since the SORT model uses the inner product of every orbit representative with the templates instead of taking the max, the number of templates is scaled down accordingly to t/k in order to end up with a final embedding dimension of t . Note that this results in the SORT embedding having fewer trainable parameters than the max-filter embedding. For the RELU network, the input data is augmented by generating the full orbit of each point, resulting in a dataset of size kn for an original dataset of size n . When G is not finite k is a user chosen parameter and the data is augmented by choosing k orbit-representatives randomly.

Distortion is then computed on this augmented dataset. Unlike the max-filter and SORT models, the RELU network is not inherently group-invariant. It treats each element of an orbit as a distinct input to be embedded. Because of this lack of invariance, pairs of points from the same orbit must be excluded when computing the distortion loss function—this is handled by the ϵ parameter. Their quotient distance is zero, and including them would send the loss to infinity unless the network is exactly group-invariant. Finally for all models gradient-norm clipping is used in order to stabilize training from wild swings that can occur when a single pair of points are strongly warped by the map. Wild swings are particularly an issue when d is small and for the SORT model since there are a smaller number of model parameters and the model tends to strongly swing each epoch. Empirically this has improved training or kept it the same in all models. When the input dimension is low or when working with matrices sometimes double precision floats must be used to avoid NaNs in the training.

2.1 Hyperparameters

The model training process inherently comes with many tunable hyperparameters which are listed here with their current values in parentheses. The current values have not been optimized and were quickly chosen to produce something that works. For now, the hyperparameters for training every model are the identical.

- `batch_size` (128): Size of mini-batch of training data on which a gradient step is performed
- `n_epochs` (100): Number of training epochs per model. epoch determines learning rate via a scheduler. The model is tested at the end of each epoch.
- `grad_steps_per_epoch` (200): Number of gradient steps to take per epoch. Each gradient step is on a new mini-batch
- `lr` (0.005): Starting learning rate
- `lr_scheduler` (CosineAnnealing): Method of decreasing the learning rate over the training period (lr decreases as a single cos curve over all epochs)

The training process is done in PyTorch using ADAM. A large batch size like 128 allows for each model to better learn the geometry of the quotient space than a smaller batch does. As low as $n = 20$ does work, but in these tests the networks with more parameters like LMF don't reach their full potential; For this reason we choose a large batch size. Finally in testing we note the best distortion found over a set number of trials (10) in order to evaluate the optimal distortion each method can attain.

2.2 Invariant Polynomials

Along with the invariant network models, each group has a corresponding invariant polynomial which can be used for embedding. The dimension of the target space varies for each group and is a fixed value for each embedding. Here I list the invariant polynomials as maps from V/G :

$$V = \mathbb{R}^d, G = \mathbb{Z}_2: f(x) = \text{vec}(xx^T) \in \mathbb{R}^{d^2}$$

$$V = \mathbb{R}^d, G = S_d: f(x)_\ell = \sum_{j=1}^d (x_j)^\ell \in \mathbb{R}^d$$

$$V = \mathbb{R}^d, G = C_d: f(x)_{i,j,k} = \begin{cases} \Re\left(\text{DFT}(x)_i \text{DFT}(x)_j \text{DFT}(x)_{i-j \pmod{d}}\right) & k = 0 \\ \Im\left(\text{DFT}(x)_i \text{DFT}(x)_j \text{DFT}(x)_{i-j \pmod{d}}\right) & k = 1 \end{cases}$$

for $i, j = 1, \dots, d, k \in \{0, 1\}$, such that $f(x) \in \mathbb{R}^{2d^2}$

$$V = \mathbb{R}^2, G = C_d: \text{Let } \mathbb{J}: \mathbb{R}^2 \rightarrow \mathbb{C} \text{ be the canonical isomorphism. Then } f(x) = \mathbb{J}^{-1}(\mathbb{J}(x)^d)$$

$$V = \mathbb{C}, G = S^1: f(z) = \text{vec}(zz^*)$$

$$V = (\mathbb{R})^p, G = O(d): f(X) = \text{vec}(XX^T)$$

3 Testing

Each epoch the models are evaluated on an independently drawn test sample. Each test sample consists of $n = 2000$ iid test points generated according to the standard Gaussian. For most of the Hilbert spaces we are working with, $2000 = \mathcal{O}(\exp|V|)$. The bottleneck for test set size is that a priori a distance matrix between all elements is needed in order to compute test distortion. At the end of each epoch the maximum distortion is computed on the test set and recorded. An

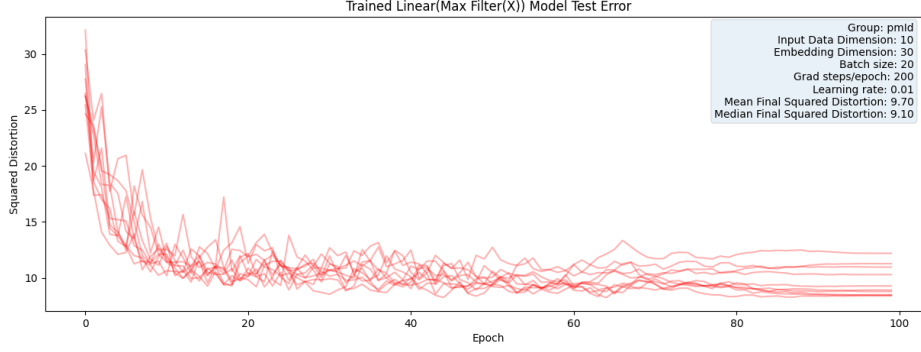


Figure 2: 10 LMF models’ test set distortions during training for embedding $\mathbb{R}^{10}/\mathbb{Z}2 \rightarrow \mathbb{R}^{30}$.

example plot of test-set distortion over the training period is shown in Figure 2. When considering the map given by random choice of a max-filter bank there is no training, instead a number of trials ($N = 2000$) are run with random templates each time and the minimum test-set distortion is recorded. For the trained models, $n_trials = 10$ independently trained copies of each models is trained and then their minimum test-set distortions are recorded. For $O(3)$, $n_trials = 1$ due to runtime. The invariant polynomials neither have randomness nor training and so their values on a given test set are fixed and independent of the other models’ embedding dimensions.

The final table has rows labeled by V/G and columns by the architectures used to train. We choose embedding dimension $t = 3|V|$. The columns of the table are labeled by the architecture of the embedding. The goal of this table is to compare the empirical distortions obtained by training each type of model to embed a variety of quotient spaces. When ReLU is used for continuous groups, $k = 10$ orbit representatives are sampled in the data augmentation. When SORT is used $t = \left(\lceil |G| \lceil \frac{3|V|}{|G|} \rceil \right)$ since the order of the group must divide the embedding dimension.

	MF	SORT	LRMF	LMF	ReLU	RMF	Poly	H. Poly
$\mathbb{R}^5/\mathbb{Z}_2 \rightarrow \mathbb{R}^{15}$	2.08	7.05	4.42	2.12	2.03	4.18	17.70	1.41
$\mathbb{R}^5/S_5 \rightarrow \mathbb{R}^{15}$	3.06	1.00	1.00	1.00	—	5.69	4.44e3	2.15e4
$\mathbb{R}^5/C_5 \rightarrow \mathbb{R}^{15}$	4.55	3.37	3.89	2.99	7.62	5.56	702	6,960
$\mathbb{R}^2/C_3 \rightarrow \mathbb{R}^6$	2.93	1.93	2.08	1.60	2.58e3	3.05	2.42e4	3.00
$\mathbb{R}^2/C_4 \rightarrow \mathbb{R}^6$	4.16	1.96	2.08	1.54	16.20	4.43	3.89e6	4.00
$\mathbb{C}^2/S^1 \rightarrow \mathbb{R}^{12}$	2.17	—	1.77	1.51	17.60	4.25	21.00	1.41
$(\mathbb{R}^3)^3/O(3) \rightarrow \mathbb{R}^{27}$	3.18	—	1.58	1.41	2.07e3	4.51	14.15	5.21

Table 1: Minimum test-set distortion for each trained embedding. Left of the double vertical line are trained methods, and to the right are untrained. For each setting, the best performing trained model is bolded.

	MF	LRMF	LMF	ReLU	RMF	Poly	H. Poly
$\mathbb{R}^2/\mathbb{Z}_2 \rightarrow \mathbb{R}^8$	1.84	2.01	1.66	1.64	1.92	163	1.41
$\mathbb{R}^2/\mathbb{Z}_2 \rightarrow \mathbb{R}^{16}$	1.67	1.54	1.46	1.63	1.78	163	1.41
$\mathbb{R}^2/\mathbb{Z}_2 \rightarrow \mathbb{R}^{32}$	1.66	1.49	1.44	1.58	1.80	163	1.41
$\mathbb{R}^2/\mathbb{Z}_2 \rightarrow \mathbb{R}^{256}$	1.66	1.42	1.42	1.51	1.76	163	1.41

Table 2: Minimum test-set distortions for each embedding of $\mathbb{R}^2/\mathbb{Z}_2$, where $\mathbb{Z}_2 \cong \langle -I_2 \rangle$.

	MF	LRMF	LMF	ReLU	RMF	Poly	H. Poly
$\mathbb{C}^2/S^1 \rightarrow \mathbb{R}^8$	2.20	1.81	1.64	15.26	2.52	16.89	1.41
$\mathbb{C}^2/S^1 \rightarrow \mathbb{R}^{16}$	2.16	1.66	1.47	7.19	2.46	16.89	1.41
$\mathbb{C}^2/S^1 \rightarrow \mathbb{R}^{32}$	2.16	1.52	1.44	5.61	2.30	16.89	1.41
$\mathbb{C}^2/S^1 \rightarrow \mathbb{R}^{256}$	2.16	1.42	1.42	3.29	2.18	16.89	1.41

Table 3: Minimum test-set distortions for each trained embedding of $\mathbb{C}^2/S^1$ where $S^1 \cong \{e^{i\theta}\}$.

	MF	LRMF	LMF	ReLU	RMF	Poly	H. Poly
$(\mathbb{R}^2)^2/O(2) \rightarrow \mathbb{R}^4$	3.23	2.15	1.41	6.32e4	3.47	47.63	8.75
$(\mathbb{R}^2)^2/O(2) \rightarrow \mathbb{R}^8$	2.55	1.46	1.40	5.23e4	2.91	47.63	8.75
$(\mathbb{R}^2)^2/O(2) \rightarrow \mathbb{R}^{16}$	2.43	1.40	1.40	1.13e5	2.77	47.63	8.75

Table 4: Minimum test-set distortions for each trained embedding of $(\mathbb{R}^2)^2/O(2)$ where $O(2)$ acts on column vectors.

3.1 Attaining Optimal Distortions

Another table is created to investigate the effect of embedding dimension on the distortion for specific group actions. We consider only $\mathbb{R}^2/\mathbb{Z}_2$, $\mathbb{C}/S^1$, and $(\mathbb{R}^2)^2/O(2)$ for this setting in order to investigate the convergence towards their known optimal metric distortions of $\sqrt{2}$. Furthermore the distortions attained by MF for $\mathbb{R}^2/\mathbb{Z}_2$ and $\mathbb{C}/S^1$ converge to the optimal values attainable by max-filtering alone. The results are shown in Tables 2, 3, and 4. For these tests a single test sample of size $n = 2,000$ is used with the same training parameters and 10 trials per embedding.

4 Visualizing Embeddings

In this section we visualize the embeddings of $\mathbb{R}^2/\mathbb{Z}_2$ created by the LMF architecture. To do this we first run the training scheme as in the previous section to obtain trained parameters defining the

embedding. With these parameters it is possible to visualize the space post-embedding by drawing iid Gaussian samples, embedding them, and then visualizing that embedding in $\mathbb{R}^3$ by its first three principal components.



Figure 3: Visualization of the embedding $\mathbb{R}^2/\mathbb{Z}_2 \xrightarrow{LMF} \mathbb{R}^{128} \xrightarrow{PCA} \mathbb{R}^2$ with Gaussian input.

LMF creates a piecewise linear map (whose pieces are defined by the max in max-filtering) and this can be visualized in the same vein by embedding only $S^1 \subset \mathbb{R}^2$ and observing the singularities in its visualization (note that post-composing by projecting onto the top 3 principal components it's still piecewise linear). The embedded pieces of S^1 can also be used to reconstruct the planes in $\mathbb{R}^3$ that are embedded onto, as is done in Figure 5. Here we see that as we increase the embedding dimension/increase number of training parameters the piecewise linear embedding better approximates the boundary of the Lorenz cone which is indeed an optimal embedding of $\mathbb{R}^2/\mathbb{Z}_2$.

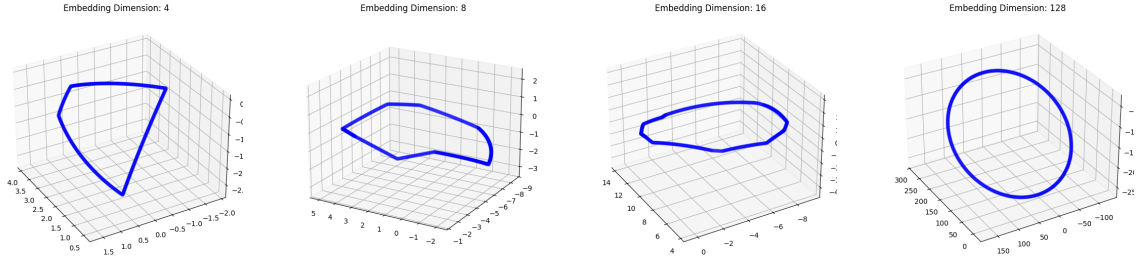


Figure 4: Embeddings of $S^1 \subset \mathbb{R}^2/\pm$ visualized using PCA for various embedding dimensions

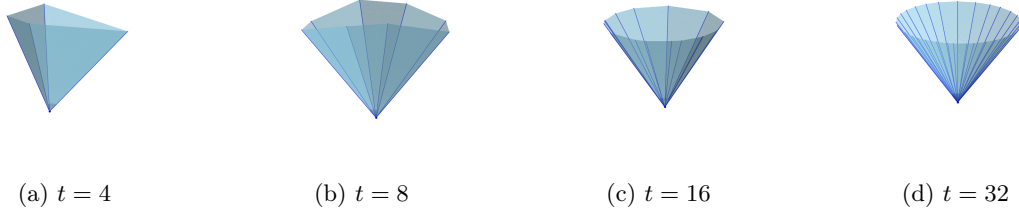


Figure 5: The embedding of $\mathbb{R}^2/\pm$ visualized in $\mathbb{R}^3$ using PCA for increasing embedding dimensions.

4.1 Visualizing Congressional Districts

In this section, rather than training using gaussian sampling we train on the 2025 119th congressional districts sampled as 50 points in $\mathbb{R}^2$ uniformly along their shape boundary. Any congressional districts that are not a single connected component are discarded as to not throw off the embedding of the space of connected districts. In doing this, 17 districts are discarded, and 427 remain to be embedded. These outlines are considered modulo rotations and the starting point of the sampling i.e. in $(\mathbb{R}^2)^{50}/(O(2) \times C_{50})$. The model is trained directly to obtain the best distortion on this set of shapes and no distinct test set is required for this application. Taking $t = 3|V|$ as in Table 1 did not lead to any empirical improvement over $t = 100$, so we choose the more efficient model size for this application. All other model parameters are kept the same as in the previous setting. The following table reports the minimum distortions attained by training 10 independent models of each type (or in the case of RMF taking the minimum over $N = 2000$ random max-filter banks).

	MF	LRMF	LMF	RELU	RMF
$(\mathbb{R}^2)^{50}/(O(2) \times C_{50}) \rightarrow \mathbb{R}^{100}$	3.73	1.81	1.50	10.51	5.39

Table 5: Minimum distortion obtained by embedding the congressional districts in shape space

Figure 6 shows the districts whose positions in the plane are given by their top two principal components after the LMF embedding. The horizontal axis seems to represent convexity of the district whereas the vertical axis represents eccentricity. This visualization gives a measure of how "oddly shaped" a district is, given by how much of an outlier it is either in the bottom or left direction. Some districts which have been criticized by news outlets as being gerrymandered (see "thefulcrum" or "statista") appear in red in the embedding visualization and all appear in the bottom-left portion.

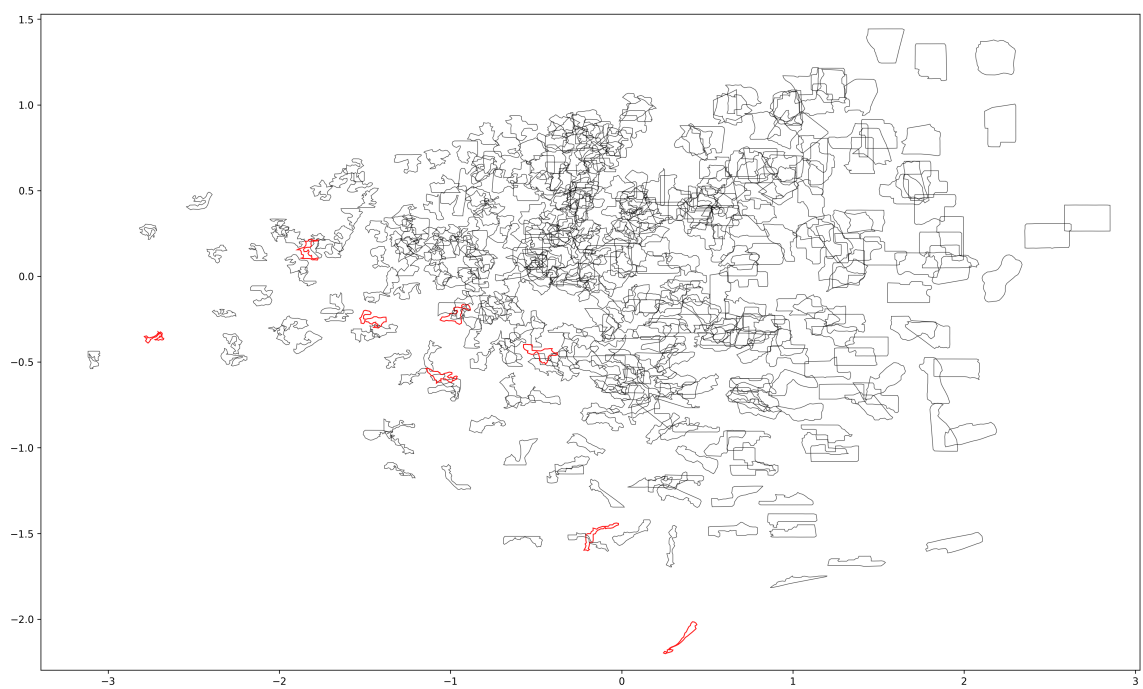


Figure 6: The US Congressional Districts consisting of a single connected component are embedded using the Linear of Max Filter map and plotted according to top 2 principal components